

Propuesta de Trabajo Práctico Integrador (TPI)

Q-Sec: Simulador Interactivo del Protocolo de Criptografía Cuántica BB84

Materia: Soporte a Gestión de Datos con Programación Visual

Grupo: 12

Integrantes: Lucio Cosentino, Tomas Wardoloff, Santino Cataldi, Gaspar Martinez.

Introducción.

En un mundo cada vez más digital, la seguridad en las comunicaciones es fundamental. Los métodos criptográficos actuales, como RSA, basan su seguridad en problemas matemáticos que son difíciles de resolver para las computadoras clásicas. Sin embargo, la llegada de las computadoras cuánticas amenaza con romper estos sistemas. La criptografía cuántica surge como una solución, ofreciendo una seguridad garantizada por las propias leyes de la física.

El protocolo BB84, propuesto en 1984 por Charles Bennett y Gilles Brassard, es el ejemplo más famoso de Distribución Cuántica de Claves (QKD). Permite a dos partes, tradicionalmente llamadas Alice y Bob, establecer una clave secreta compartida con la capacidad de detectar a cualquier espía, llamado Eve, que intente interceptar la comunicación.

Este proyecto, Q-Sec, propone el desarrollo de un sistema web educativo que simula el protocolo BB84. El objetivo es desmitificar la criptografía cuántica, permitiendo a los usuarios ejecutar una simulación paso a paso, visualizar los resultados y entender empíricamente cómo la mecánica cuántica hace posible una comunicación segura y la detección de intrusos.

Abstract.

El presente trabajo detalla el diseño y la implementación de Q-Sec, una plataforma de software desarrollada en Python bajo una estricta arquitectura de capas. El sistema tiene como finalidad simular el protocolo de distribución cuántica de claves BB84, proporcionando una herramienta interactiva y educativa. La capa de presentación se construye con el framework web Flask, mientras que la lógica de negocio utiliza la librería Qiskit de IBM para simular la preparación, transmisión y medición de qubits. La persistencia de los datos, como los usuarios y los resultados de cada sesión de intercambio, se gestiona mediante el ORM SQLAlchemy sobre una base de datos SQLite. El sistema implementa reglas de negocio críticas, como la detección de espionaje basada en la tasa de error cuántico, cuya correcta implementación es validada a través de tests unitarios con Pytest. El resultado es una aplicación web funcional que demuestra la integración de simulaciones cuánticas en un entorno de software tradicional y robusto.

Requerimientos Funcionales (RF).

RF01: El sistema debe permitir el registro y autenticación de usuarios.

RF02: Un usuario autenticado debe poder iniciar una nueva "Sesión de Intercambio de Clave".

RF03: Al iniciar una sesión, el usuario debe poder configurar sus parámetros: la longitud de la clave inicial (ej. 64 bits) y si desea incluir un espía ("Eve") en la simulación.

RF04: El sistema debe ejecutar la simulación del protocolo BB84 completo:

- a. Generación de bits y bases aleatorias por parte de Alice.
- b. Codificación de los bits en qubits usando Qiskit.
- c. (Opcional) Interceptación y reenvío de qubits por parte de Eve.
- d. Generación de bases aleatorias y medición de qubits por parte de Bob.
- e. Comparación pública de bases para filtrar la clave.
- f. Verificación de la tasa de error para detectar a Eve.

RF05: El sistema debe almacenar en la base de datos el resultado de cada sesión: la configuración, si la clave fue segura o comprometida, y la clave final (si es segura).

RF06: El usuario debe poder ver un historial de sus sesiones de intercambio pasadas con sus resultados.

Requerimientos No Funcionales (RNF).

RNF01 (Rendimiento): La simulación completa para una clave de 128 bits debe finalizar en menos de 30 segundos.

RNF02 (Seguridad): Las contraseñas de los usuarios deben ser almacenadas en la base de datos utilizando una función de hash (ej. Werkzeug security helpers).

Stack Tecnológico.

Lenguaje de Programación: Python 3.9+

Arquitectura: En capas (Presentación, Negocio, Datos).

Framework Web (Capa Presentación): Flask.

Simulación Cuántica (Capa Negocio): IBM Qiskit.

Base de Datos (Capa Datos): SQLite.

ORM (Capa Datos): SQLAlchemy.

Testing (Capa Negocio): Pytest.

Control de Versiones: Git / GitHub.

Reglas de Negocio (RN).

RN01: Detección de Espionaje por Tasa de Error: En una simulación donde "Eve" está presente, se utiliza una porción de la clave filtrada como muestra de control.